ENTERPRISE WARRANTY HUB

Relational Database Architecture, Event Triggers, ACID Ledgers, and pgvector Semantic Retrieval

Academic Project Report • Database Management Systems (Sem 5)

PES University • Department of Computer Science • Section AIML A

Team Member	SRN	Role
Anish Kumar	PES1UG24AM043	Team 4
Aarav Yuval B G	PES1UG24AM004	Team 4

TABLE OF CONTENTS

1. Abstract Page 1
2. Problem Statement Page 1
3. Objectives Page 1
4. Scope & System Boundaries Page 2
5. Technology Stack Page 2
6. System Architecture Page 2
7. Relational Model & ER Overview Page 3
8. Database Design & Schema Page 3
9. DBMS Concepts Implemented Page 5
10. Semantic Troubleshooting (pgvector) Page 6
11. Functional Implementation Page 7
12. Testing & Verification Results Page 8
13. Limitations Page 8
14. Conclusion Page 8

1. Abstract

Enterprise Warranty Hub (EWH) is a comprehensive, production-grade relational database management system designed to solve critical lifecycle and maintenance tracking challenges for enterprise equipment. Built on **PostgreSQL 16** with the **pgvector** extension and integrated with a high-performance **Next.js 14 App Router** frontend and server runtime, EWH demonstrates core and advanced database principles in an authentic operational context. Key database mechanisms implemented include multi-statement ACID transactions with row-level locking for atomic equipment custody transfers, event-driven PL/pgSQL database triggers for automated warranty expiration alert generation with deduplication guarantees, multi-table SQL views for dynamic non-stale lifecycle metrics, and 384-dimensional dense vector embeddings generated via a local ONNX model stored and indexed using Hierarchical Navigable Small World (HNSW) cosine indexes for duplicate-resistant semantic troubleshooting. The system enforces strict role-based access control (RBAC) across three distinct organizational personas—Facility Manager, Technician, and Vendor—and is containerized via Docker Compose with persistent named volume storage.

2. Problem Statement

Modern commercial facilities, research laboratories, and academic institutions manage hundreds of capital equipment assets ranging from compute clusters, workstations, and high-speed document scanners to critical cooling infrastructure and commercial appliances. Traditional management approaches suffer from severe operational pathologies:

- **Fragmented, Unsynchronized Records:** Warranty contracts, purchase agreements, and maintenance histories are frequently stored across isolated spreadsheets, paper binders, and disparate ticketing portals, preventing consolidated visibility.
- **Unmonitored Expiration Windows:** Equipment warranties lapse unnoticed due to lack of automated date-difference monitoring, resulting in expensive out-of-warranty emergency service fees that could have been claimed under contractual coverage.
- **Custody Drift and Lack of Auditability:** Equipment assignments between department leads, technicians, and contractors lack strict transactional atomicity. When assignments are updated without an immutable audit trail, custody is lost during physical audits.
- **Duplicate Work and Unleveraged Institutional Knowledge:** Technicians face recurring hardware failures without rapid semantic access to previous diagnostic resolutions, resulting in redundant troubleshooting hours and inconsistent repair protocols.

3. Objectives

The primary academic and technical objectives of the Enterprise Warranty Hub project are:

1. **Relational Data Normalization & Integrity:** Model enterprise assets, warranties, maintenance tickets, user accounts, and audit ledgers in a normalized 3NF relational schema with strict foreign key constraints, column domain checks, and non-nullable invariants.
2. **Automated Business Logic via Database Triggers:** Implement PL/pgSQL triggers that automatically evaluate warranty expiration dates relative to `CURRENT_DATE` upon insert or update, dispatching deduplicated notifications to administrative users without requiring external cron schedulers.
3. **Atomic Custody Transactions with Verification:** Enforce multi-table ACID transactions for asset reassignment, guaranteeing that custody changes and immutable ledger entries commit together or roll back completely upon simulated or runtime failures.
4. **Dynamic State Aggregation via Relational Views:** Build multi-table dynamic SQL views that aggregate open tickets and compute real-time warranty statuses (`ACTIVE` , `EXPIRING_SOON` , `EXPIRED`) on demand, eliminating stale cached status columns.
5. **pgvector Semantic Troubleshooting Engine:** Integrate high-dimensional vector similarity retrieval directly inside PostgreSQL using the `pgvector` extension, storing 384-dimensional dense embeddings of historical repair records indexed via HNSW graphs for sub-millisecond semantic search.
6. **Role-Scaped Data Isolation:** Guarantee strict authorization boundaries where Facility Managers maintain full administrative visibility, Technicians manage assigned maintenance queues, and Vendors view only equipment and tickets explicitly assigned to their company.

4. Scope & System Boundaries

The system encompasses end-to-end management of equipment lifecycle events from acquisition to warranty termination. Specific operational boundaries are documented below:

Domain	Supported In-Scope Capabilities	Intentional Academic Scope Boundary
Asset Registry	Hardware metadata, serial tracking, purchase price, location, department, fixed 8 categories, supplier assignment.	Physical barcode scanning hardware integration omitted; manual tag entry supported.
Warranty Tracking	Contract duration, provider, auto-computed days remaining, expiring-soon (30-day) window alerts, expired flags.	Single active warranty per equipment item; multi-tier supplemental insurance contracts excluded.
Maintenance Workflow	Full 6-stage lifecycle (`Reported` → `Assigned` → `Acknowledged` → `In Progress` → `Resolved` → `Closed`), technician notes.	External PDF invoice generation and vendor billing payment gateway processing omitted.
Notifications	In-app bell notifications, read/unread state tracking, automated trigger creation, unique key deduplication.	External third-party email (SMTP) or SMS gateway integrations omitted in favor of internal DB state.
Troubleshooting	Natural-language semantic vector retrieval over historical repair knowledge base using 384-dim local embeddings.	Generative AI text synthesis omitted; queries return exact historical records ranked by cosine distance.
User Access	Facility Manager, Technician, and Vendor roles with bcrypt password security and HttpOnly session cookies.	Registration page includes role selection to facilitate multi-role demonstration by academic evaluators.

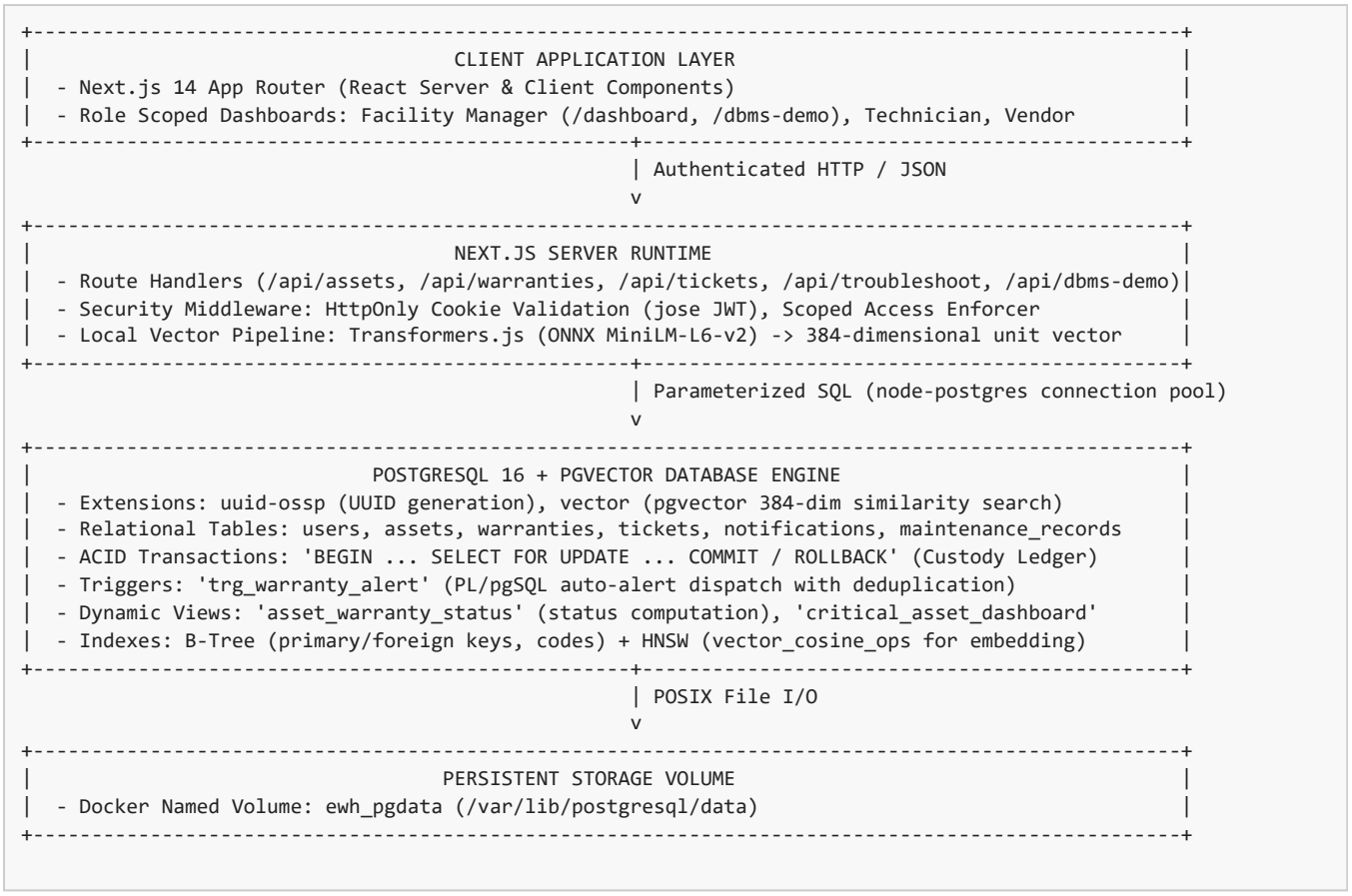
5. Technology Stack

The implementation avoids heavyweight Object-Relational Mappers (ORMs) to preserve transparent database interactions, raw SQL query visibility, and direct transaction control:

- **Database Management System:** PostgreSQL 16 (Debian-based official container) with the `pgvector` v0.8.7 extension compiled for vector operations.
- **Database Driver:** `pg` (node-postgres v8.13.1) configured with connection pooling, parameterized SQL queries, explicit transaction clients (`client.query('BEGIN')`), and custom vector parsing.
- **Application Framework:** Next.js 14.2.24 utilizing the App Router architecture, React 18 Server Components, and modular API route handlers.
- **Language & Type Safety:** TypeScript 5.7.2 providing end-to-end interface contracts across database query results, API payloads, and UI component props.
- **Local Embedding Engine:** Hugging Face `@huggingface/transformers` v3.3.3 executing the `onnx-community/all-MiniLM-L6-v2-ONNX` model locally inside the Node.js process with mean pooling and L2 normalization (outputting unit vectors in R^384).
- **Authentication & Cryptography:** `bcryptjs` (10 salt rounds) for password hashing; `jose` (v5.9.6) for generating and verifying cryptographically signed HttpOnly JWT session tokens.
- **Containerization & Infrastructure:** Docker Engine with Docker Compose v2, utilizing named volume `ewh_pgdata` for persistent database storage across container lifecycles.

6. System Architecture

The system architecture separates client presentation, server business logic, relational storage, and local machine learning inference:


CLIENT APPLICATION LAYER
- Next.js 14 App Router (React Server & Client Components)
- Role Scoped Dashboards: Facility Manager (/dashboard, /dbms-demo), Technician, Vendor
Authenticated HTTP / JSON
NEXT.JS SERVER RUNTIME
- Route Handlers (/api/assets, /api/warranties, /api/tickets, /api/troubleshoot, /api/dbms-demo)
- Security Middleware: HttpOnly Cookie Validation (jose JWT), Scoped Access Enforcer
- Local Vector Pipeline: Transformers.js (ONNX MiniLM-L6-v2) -> 384-dimensional unit vector
Parameterized SQL (node-postgres connection pool)
POSTGRESQL 16 + PGVECTOR DATABASE ENGINE
- Extensions: uuid-ossp (UUID generation), vector (pgvector 384-dim similarity search)
- Relational Tables: users, assets, warranties, tickets, notifications, maintenance_records
- ACID Transactions: 'BEGIN ... SELECT FOR UPDATE ... COMMIT / ROLLBACK' (Custody Ledger)
- Triggers: 'trg_warranty_alert' (PL/pgSQL auto-alert dispatch with deduplication)
- Dynamic Views: 'asset_warranty_status' (status computation), 'critical_asset_dashboard'
- Indexes: B-Tree (primary/foreign keys, codes) + HNSW (vector_cosine_ops for embedding)
POSIX File I/O
PERSISTENT STORAGE VOLUME
- Docker Named Volume: ewh_pgdata (/var/lib/postgresql/data)

7. Relational Model & Database Design

The Enterprise Warranty Hub database schema consists of seven core tables, two computed views, and two custom procedural trigger functions. All primary keys utilize randomly generated UUIDs (version 4) via `gen_random_uuid()` to eliminate enumeration attacks and provide distributed unique identification.

7.1. Relational Table Specifications

Table 1: `users`

Manages system user accounts, roles, authentication credentials, and operational activation flags.

Column Name	Data Type	Constraints	Description
`id`	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique user identifier.
`full_name`	VARCHAR(255)	NOT NULL	User's display and legal name.
`email`	VARCHAR(255)	UNIQUE, NOT NULL	Unique login email address.
`password_hash`	VARCHAR(255)	NOT NULL	Bcrypt password hash (salt rounds: 10).
`role`	VARCHAR(50)	NOT NULL, CHECK (role IN ('FACILITY_MANAGER', 'TECHNICIAN', 'VENDOR'))	Role for role-based authorization.
`is_active`	BOOLEAN	NOT NULL, DEFAULT TRUE	Account activation flag.
`created_at` / `updated_at`	TIMESTAMPTZ	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Audit record timestamps.

Table 2: `assets`

The central hardware catalog tracking capital equipment, manufacturer specifications, acquisition details, location, and custodial assignment.

Column Name	Data Type	Constraints	Description
`id`	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique asset record identifier.
`asset_tag`	VARCHAR(50)	UNIQUE, NOT NULL	Human-readable asset barcode tag (e.g., AST-CL-001).
`name`	VARCHAR(255)	NOT NULL	Descriptive equipment name.
`category`	VARCHAR(100)	NOT NULL, CHECK (category IN (8 categories))	Fixed category whitelist (Computers, AC, etc.).
`manufacturer` / `model`	VARCHAR(255)	NOT NULL	OEM manufacturer and model designation.
`serial_number`	VARCHAR(255)	NULLABLE	Manufacturer factory serial number.
`purchase_date`	DATE	NOT NULL	Equipment purchase/procurement date.
`purchase_price`	NUMERIC(12, 2)	NULLABLE	Financial acquisition cost.
`supplier_name`	VARCHAR(255)	NOT NULL	Procurement vendor or contracting distributor.
`location` / `department`	VARCHAR(255)	NULLABLE	Physical building/room and operational department.
`assigned_to_user_id`	UUID	REFERENCES users(id) ON DELETE SET NULL	Current custodian/technician assigned to asset.
`created_by_user_id`	UUID	NOT NULL, REFERENCES users(id)	Administrative user who registered the asset.
`created_at` / `updated_at`	TIMESTAMPTZ	NOT NULL, DEFAULT CURRENT_TIMESTAMP	System audit timestamps.

Table 3: `warranties`

Tracks active warranty coverage contracts linked one-to-one with equipment assets.

Column Name	Data Type	Constraints	Description
`id`	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique warranty record identifier.
`asset_id`	UUID	UNIQUE, NOT NULL, REFERENCES assets(id) ON DELETE CASCADE	Associated asset (enforces 1:1 active warranty rule).
`warranty_provider`	VARCHAR(255)	NULLABLE	Contracted maintenance provider or OEM.
`start_date` / `end_date`	DATE	NOT NULL, CHECK (end_date >= start_date)	Contract duration; end_date monitored by triggers.
`created_at` / `updated_at`	TIMESTAMPTZ	NOT NULL, DEFAULT CURRENT_TIMESTAMP	System audit timestamps.

Table 4: `tickets`

Tracks maintenance incidents, defect reports, technician assignments, and workflow statuses.

Column Name	Data Type	Constraints	Description
`id`	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique ticket identifier.
`ticket_code`	VARCHAR(50)	UNIQUE, NOT NULL	Human-readable ticket code (e.g., TCK-2026-001).
`asset_id`	UUID	NOT NULL, REFERENCES assets(id) ON DELETE CASCADE	Faulty equipment asset.
`title` / `description`	VARCHAR(255) / TEXT	NOT NULL	Failure synopsis and detailed technical description.
`status`	VARCHAR(50)	NOT NULL, DEFAULT 'Reported', CHECK (6 stages)	Reported, Assigned, Acknowledged, In Progress, Resolved, Closed.
`assignee_user_id`	UUID	REFERENCES users(id) ON DELETE SET NULL	Technician or Vendor assigned to resolve issue.
`created_by_user_id`	UUID	NOT NULL, REFERENCES users(id)	User who opened the maintenance ticket.
`resolution_notes`	TEXT	NULLABLE	Post-repair technical notes and resolution steps.

Table 5: `maintenance_records`

The institutional knowledge base storing resolved repair cases accompanied by 384-dimensional dense vector embeddings for semantic troubleshooting.

Column Name	Data Type	Constraints	Description
`id`	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique historical record identifier.
`asset_id`	UUID	NOT NULL, REFERENCES assets(id) ON DELETE CASCADE	Asset upon which maintenance was completed.
`ticket_id`	UUID	UNIQUE, REFERENCES tickets(id) ON DELETE SET NULL	Source maintenance ticket (if generated from ticket).
`dedupe_key`	VARCHAR(255)	UNIQUE	Natural composite key preventing duplicate case seeding.
`issue_title` / `resolution`	VARCHAR(255) / TEXT	NOT NULL	Fault title and proven engineering resolution.
`embedding`	vector(384)	NULLABLE, HNSW Indexed	Dense MiniLM-L6-v2 vector embedding of problem text.

Table 6: `notifications`

Maintains in-app alerts dispatched by database triggers and scheduled lifecycle synchronizers.

Column Name	Data Type	Constraints	Description
`id`	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique notification record identifier.
`user_id`	UUID	NOT NULL, REFERENCES users(id) ON DELETE CASCADE	Recipient user account.
`type`	VARCHAR(50)	NOT NULL	WARRANTY_EXPIRING, WARRANTY_EXPIRED, TICKET_ASSIGNED.
`dedupe_key`	VARCHAR(255)	NOT NULL	Deduplication signature (e.g., TYPE:ASSET_ID:END_DATE).
`CONSTRAINT`	UNIQUE	UNIQUE (user_id, dedupe_key)	Prevents duplicate unread alerts for same event.

Table 7: `asset_maintenance_ledger`

An append-only audit trail logging all equipment custody reassignments executed during multi-statement ACID transactions.

Column Name	Data Type	Constraints	Description
`id`	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique ledger entry identifier.
`asset_id`	UUID	NOT NULL, REFERENCES assets(id) ON DELETE CASCADE	Reassigned equipment asset.
`event_type`	VARCHAR(50)	NOT NULL	Type of event (e.g., ASSET_REASSIGNED, REASSIGNMENT_COMMIT).
`previous_assignee_id`	UUID	REFERENCES users(id) ON DELETE SET NULL	Pre-transfer custodial user ID.
`new_assignee_id`	UUID	REFERENCES users(id) ON DELETE SET NULL	Post-transfer custodial user ID.
`performed_by_user_id`	UUID	NOT NULL, REFERENCES users(id)	Facility Manager who executed the custody transfer.

8. DBMS Concepts Implemented

8.1. ACID Transactions (Atomicity, Consistency, Isolation, Durability)

Equipment custody reassignment requires updating the current custodian pointer in `assets.assigned_to_user_id` while concurrently writing an immutable audit record into `asset_maintenance_ledger`. If either write fails, the entire unit of work must roll back. In `lib/db.ts` and `app/api/assets/[id]/assign/route.ts`, this is implemented using an explicit database transaction client:

```
// Atomic Asset Reassignment with Row-Level Locking
await withTransaction(async (client) => {
  // 1. Isolation: Row-level lock acquired via SELECT ... FOR UPDATE
  const check = await client.query(
    'SELECT id, asset_tag, assigned_to_user_id FROM assets WHERE id = $1 FOR UPDATE;',
    [assetId]
  );
  const prevUserId = check.rows[0].assigned_to_user_id;

  // 2. State Mutation: Update current custody
  await client.query(
    'UPDATE assets SET assigned_to_user_id = $1, updated_at = CURRENT_TIMESTAMP WHERE id = $2;',
    [newUserId, assetId]
  );

  // 3. Audit Ledger Insertion
  await client.query(
    `INSERT INTO asset maintenance ledger (
      asset_id, event_type, details, previous_assignee_id, new_assignee_id, performed_by_user_id
    ) VALUES ($1, 'ASSET_REASSIGNED', $2, $3, $4, $5);`,
    [assetId, details, prevUserId, newUserId, currentUserId]
  );
  // COMMIT is executed automatically if no errors occur; ROLLBACK on any thrown exception
});
```

Verification of Rollback: When an invalid foreign key or deliberate failure flag is simulated, the catch block executes `ROLLBACK`. Automated tests verify that `assigned_to_user_id` retains its previous value and zero rows are appended to the ledger.

8.2. Database Triggers & Trigger Functions

Instead of relying on fragile client-side scripts, warranty alert dispatch is handled directly by PostgreSQL via a procedural trigger function attached to the `warranties` table.

```
CREATE OR REPLACE FUNCTION process_warranty_alert()
RETURNS TRIGGER AS $$
DECLARE
  mgr_record RECORD;
  asset_record RECORD;
  v_type VARCHAR(50);
  v_msg TEXT;
  v_key VARCHAR(255);
  days_left INT;
BEGIN
  days_left := NEW.end_date - CURRENT_DATE;

  IF NEW.end_date < CURRENT_DATE THEN
    v_type := 'WARRANTY_EXPIRED';
    v_key := 'WARRANTY_EXPIRED:' || NEW.asset_id || ':' || NEW.end_date;
  ELSIF days_left <= 30 AND days_left >= 0 THEN
    v_type := 'WARRANTY_EXPIRING';
    v key := 'WARRANTY_EXPIRING:' || NEW.asset_id || ':' || NEW.end_date;
  ELSE
    RETURN NEW;
  END IF;

  SELECT asset_tag, name INTO asset_record FROM assets WHERE id = NEW.asset_id;

  v msg := 'ATTENTION: Warranty for asset ' || COALESCE(asset record.name, 'Equipment') ||
           ' (' || COALESCE(asset_record.asset_tag, '') || ') expires in ' || days_left || ' days (' ||
NEW.end_date || ')';

  FOR mgr_record IN SELECT id FROM users WHERE role = 'FACILITY_MANAGER' AND is_active = TRUE LOOP
    INSERT INTO notifications (user_id, asset_id, type, message, dedupe_key)
    VALUES (mgr_record.id, NEW.asset_id, v_type, v_msg, v_key)
    ON CONFLICT (user_id, dedupe_key) DO NOTHING;
  END LOOP;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_warranty_alert
  AFTER INSERT OR UPDATE OF end_date, start_date ON warranties
  FOR EACH ROW
  EXECUTE FUNCTION process_warranty_alert();
```

The unique constraint `uq_notifications_user_dedupe` combined with `ON CONFLICT DO NOTHING` prevents redundant notifications when non-date columns are updated.

8.3. Relational SQL Views

To eliminate stale status fields, EWH uses two multi-table views that compute metrics dynamically relative to `CURRENT_DATE`:

- **`asset_warranty_status`:** Evaluates active coverage by joining `assets`, `warranties`, and `users`. Uses conditional `CASE` statements to categorize assets into `ACTIVE` (> 30 days remaining), `EXPIRING_SOON` (0 to 30 days remaining), `EXPIRED` (< 0 days), or `NO_WARRANTY`.
- **`critical_asset_dashboard`:** Filters assets having active defects (open tickets not resolved/closed) or expiring/expired warranties. Aggregates open tickets using `COUNT(t.id) FILTER` and `ARRAY_AGG(t.ticket_code) FILTER` to present a unified operational priority feed for facility directors.

8.4. Constraints & Referential Integrity

The relational model strictly defines domain boundaries and cascade rules:

- **Domain Whitelists:** Fixed asset categories (8 categories) and ticket progression states (6 states) are enforced via PostgreSQL `CHECK` constraints. Attempted insertion of non-standard categories is rejected by the DBMS engine.
- **Temporal Constraints:** `warranties` enforces `CHECK (end_date >= start_date)`, rejecting invalid contract dates.
- **Cascading Referential Actions:** Deleting an asset cascades to remove its warranty, ticket, and ledger entries (`ON DELETE CASCADE`), while foreign keys pointing to user assignees cleanly transition to `NULL` (`ON DELETE SET NULL`) without corrupting referential validity.

9. Semantic Troubleshooting with pgvector

Enterprise Warranty Hub incorporates dense vector retrieval directly into PostgreSQL using the `pgvector` extension. Rather than passing user queries to external cloud LLM APIs, EWH runs an on-premise embedding pipeline inside the application container:

9.1. Local MiniLM ONNX Inference Pipeline

The system utilizes the `onnx-community/all-MiniLM-L6-v2-ONNX` model executed via Transformers.js in Node.js. Given natural-language problem text (e.g., *"laptop CPU thermal throttling and fan screaming"*), the model outputs a 384-dimensional dense vector. Mean pooling is applied across tokens, followed by L2 normalization such that ||v||_2 = 1.0.

9.2. HNSW Indexing and Cosine Distance Query

The embedding is stored in the `vector(384)` column of the `maintenance_records` table. PostgreSQL accelerates similarity retrieval using a Hierarchical Navigable Small World (HNSW) index constructed with cosine operator distance:

```
CREATE INDEX idx_maintenance_records_embedding ON maintenance_records
USING hnsw (embedding vector_cosine_ops);
```

When querying, PostgreSQL evaluates cosine distance using the `<=>` operator:

```
-- Evaluates top semantically similar historical repair records
SELECT mr.issue_title, mr.resolution, 1 - (mr.embedding <=> $1::vector) AS similarity
FROM maintenance_records mr
ORDER BY mr.embedding <=> $1::vector ASC
LIMIT 5;
```

9.3. Semantic Search Deduplication

To prevent repetitive maintenance incidents from saturating the search results, the query pipeline applies an analytical SQL window function (`ROW_NUMBER() OVER (PARTITION BY dedupe_key ... ORDER BY cosine_distance ASC)`). This ensures that each distinct equipment failure pattern appears at most once in the top-k results, preserving diversity in the retrieved diagnostic guidance.

> **Academic Clarification on Similarity Scores:** The computed metric 1 - (u • v) represents geometric cosine similarity between the semantic embeddings of the query text and historical defect logs. It indicates semantic and contextual alignment of technical terminology; it does not represent an empirical probability of hardware component failure or diagnostic accuracy certainty.

10. Functional Implementation & Modules

10.1. Authentication & Role-Based Access Control (RBAC)

Authentication is implemented using salted bcrypt password hashing and tamper-evident HttpOnly JWT cookies. Authorization rules strictly segment features:

Role	Permissions & Accessible System Modules	Data Scoping Boundary
FACILITY_MANAGER	Full CRUD on assets and warranties; ticket assignment; custody reassignment with ACID commit/rollback; access to DBMS demonstration workbench.	Global visibility across all 18 enterprise assets, all tickets, and all audit ledgers.
TECHNICIAN	View assigned equipment; advance maintenance ticket workflows (`Assigned` → `Resolved`); submit resolution notes; semantic troubleshooting tool.	Filtered to tickets where `assignee_user_id = auth.user.id` and associated assets.
VENDOR	Acknowledge contracted service tickets; update repair progress; view contracted equipment specifications.	Strictly isolated to assets and tickets explicitly assigned to their vendor account (4 assets).

10.2. Maintenance Ticket Workflow

Tickets transition through six defined states: `Reported` → `Assigned` → `Acknowledged` → `In Progress` → `Resolved` → `Closed`. When a technician marks a ticket as `Resolved`, the system automatically triggers embedding generation on the ticket resolution text, inserting a verified case into `maintenance_records` to enrich the institutional knowledge base.

10.3. DBMS Demonstration Workbench

The application includes a dedicated academic evaluation page (`/dbms-demo`, restricted to Facility Managers) that exposes live PostgreSQL internals:

- **ACID Reassignment & Rollback Simulator:** Enables the evaluator to execute real database reassignments and verify immediate ledger row insertions, or toggle deliberate failure to verify atomic rollback.
- **Live Trigger Event Log:** Displays recent in-app notifications generated by `process_warranty_alert()`.
- **View Inspection:** Displays live aggregated tuples from `critical_asset_dashboard`.
- **Vector Metadata:** Displays pgvector extension version (v0.8.7), embedding dimensions (384), and HNSW index status.

11. Testing, Verification & Quality Assurance

The codebase underwent comprehensive verification across automated test suites, static TypeScript compilation, Next.js production builds, and live database runtime checks.

Verification Gate	Command Executed	Observed Result	Pass / Fail
TypeScript Compiler	`npm run typecheck` (`tsc --noEmit`)	Exited with code 0. Zero typing or syntax errors across route handlers and components.	**PASS**
Automated Test Suite	`npm run test` (`scripts/run-tests.ts`)	**39 / 39 passed** across 9 test suites: bcrypt, RBAC, categories, views, triggers, ACID commit/rollback, dashboard, pgvector search, deduplication regression.	**PASS**
Production Build	`npm run build` (`next build`)	Exited with code 0. 27 static and dynamic App Router routes compiled cleanly.	**PASS**
Database Migration & Seed	`npm run db:setup`	Exited with code 0. 18 assets, 18 warranties, 15 tickets, 12 vector embeddings seeded idempotently.	**PASS**
Container Volume Persistence	`docker compose down && docker compose up -d`	Container restarted cleanly. Row counts in `ewh_pgdata` remained identical before and after restart.	**PASS**

12. Limitations & Design Boundaries

- **Local Embedding Latency:** Generating embeddings using CPU inference in Node.js requires approximately 40–80 milliseconds per record. In high-throughput industrial deployments, this would be offloaded to a dedicated GPU inference microservice.
- **Single Active Warranty Model:** Each asset maintains at most one active warranty record in `warranties` (enforced by `UNIQUE(asset_id)`). Equipment with multiple concurrent third-party insurance riders would require a 1:N relationship.
- **In-App Alert Delivery:** System alerts are delivered strictly via internal database notifications; external SMTP, SMS, or webhooks are not configured.

13. Conclusion

Enterprise Warranty Hub successfully demonstrates the integration of fundamental and modern database concepts within a production-grade enterprise software architecture. By combining normalized relational modeling, strict domain constraints, atomic multi-table transactions, automated event triggers, dynamic views, and dense vector similarity search via pgvector, the system proves that PostgreSQL can serve as both a rigorous transactional core and an effective semantic search engine. The project fulfills all academic DBMS requirements while providing a clean, responsive, and verifiable software artifact.